

HelmNet

Safety Helmet Classification



INTRODUCTION TO COMPUTER VISION

Sarah Wolberg | July 26, 2026

4,125

labelled images

4

models compared

92.5%

best validation accuracy

Contents / Agenda

- **Executive Summary** — 3–4
- **Business Problem Overview** — 5
- **Solution Approach** — 6
- **Data Overview** — 7–10
- **Exploratory Data Analysis** — 11–15
- **Data Pre-processing** — 16–20
- **Model Building** — 21–22
- **Model 1: CNN from Scratch** — 23–29
- **Model 2: VGG16 Base** — 30–35
- **Model 3: VGG16 + FFNN** — 36–41
- **Model 4: VGG16 + FFNN + Augmentation** — 42–49
- **Model Comparison and Final Selection** — 50–54
- **Data Background and Contents** — 55
- **Actionable Insights and Recommendations** — 56–62
- **Final Conclusion** — 63

Executive Summary

SafeGuard Corp commissioned an automated image-classification system to determine whether workers on construction sites and industrial plants are wearing safety helmets. Manual monitoring of helmet compliance is slow, inconsistent between observers, and difficult to scale, and every missed violation leaves a worker exposed to a preventable head injury. The project delivers a model that classifies a worker image into one of two categories: With Helmet or Without Helmet.

The dataset comprised 4,125 labelled images captured under varied lighting, camera angles, distances, and worker activities. The two classes are imbalanced — 3,161 helmet images (76.6%) against 964 no-helmet images (23.4%) — so a model that simply predicted “helmet” for every image would already appear 76.6% accurate. Evaluation was therefore centred on macro recall, macro F1-score, and no-helmet recall rather than headline accuracy. Images were split into 3,300 training, 412 validation, and 413 test examples at 200×200 resolution and scaled to a 0–1 pixel range.

Executive Summary (continued)

Four models were trained and compared: a convolutional neural network built from scratch, a frozen VGG16 base, VGG16 with a feed-forward classification head, and that same architecture trained on augmented images. All four reached roughly 92% validation accuracy, but Model 4 — VGG16 with a feed-forward head and data augmentation — produced the most balanced result, with 92.5% accuracy, 88.9% macro recall, and an 89.4% macro F1-score, and was selected for final test evaluation.

On the held-out test set the model achieved 89.1% accuracy, 85.4% macro recall, and an 85.0% macro F1-score — comfortably above the 76.6% majority-class baseline, though three to five points below validation. Performance remained uneven across classes: 92.4% of helmet images were classified correctly against only 78.4% of no-helmet images, meaning 21 of 97 unsafe test cases were labelled as compliant.

The recommendation is to deploy the model as a decision-support tool with human review rather than as an autonomous enforcement system, and to prioritise collecting additional no-helmet imagery, tuning the decision threshold on validation data, and monitoring no-helmet recall after deployment.

Business Problem Overview

Workplace safety in hazardous environments such as construction sites and industrial plants depends heavily on consistent use of protective equipment, and the safety helmet is the single most important control against head injuries caused by falling objects and machinery. Non-compliance materially raises the risk of serious injury or fatality, yet enforcement still depends largely on supervisors noticing violations in person.

That manual oversight does not scale. Across large operations with hundreds of workers, continuous visual checking is impractical, inconsistent between observers, and prone to error — and violations are most likely to be missed exactly when sites are busiest and risk is highest.

SafeGuard Corp therefore set out to automate helmet monitoring through image analysis. The task was framed as a binary classification problem: given an image of a worker, determine whether that worker is With Helmet or Without Helmet, so that compliance can be measured continuously and unsafe conditions flagged for intervention rather than discovered after an incident.

Solution Approach

The 4,125 images were loaded at 200×200 resolution in three colour channels, normalised to a 0–1 pixel range, and divided into 3,300 training, 412 validation, and 413 test examples. Because roughly three-quarters of the data belongs to the helmet class, the evaluation framework was defined up front around macro recall, macro F1-score, and confusion matrices, with no-helmet recall treated as the safety-critical measure.

Four architectures were developed in sequence. A convolutional network trained from scratch established the baseline. Transfer learning followed, using the pretrained VGG16 convolutional base with frozen weights — first with a simple classifier, then with a feed-forward head of 64 and 32 units and 30% dropout. The final variant trained that same architecture on augmented images, applying 15-degree rotations, 10% width, height, shear and zoom shifts, and horizontal flips to widen the effective training variety.

The four models were compared on validation performance, and the strongest configuration was then carried forward to a single evaluation on the untouched test set.

Data Overview

The HelmNet dataset contains 4,125 labelled images of workers in hazardous environments, divided into two classes: label 1, With Helmet, covering 3,161 images, and label 0, Without Helmet, covering 964. Random sampling from both classes confirmed that images and labels were correctly aligned before modelling began.

The classes are markedly imbalanced — 76.6% helmet against 23.4% no-helmet, roughly 3.28 times as many compliant as non-compliant examples. This sets a deceptively high floor for accuracy: a model predicting With Helmet for every image would score about 76.6% while failing to detect a single unsafe worker.

That imbalance shaped three decisions: a stratified split so every subset preserves the original class proportions; evaluation built on macro precision, macro recall, macro F1-score and the confusion matrix rather than accuracy alone; and recall on the Without Helmet class treated as the safety-critical measure, since standard binary recall treats label 1 as positive and so reports helmet detection rather than unsafe cases.

Data Overview: Loading the Data

Loaded from the HelmNet_Project folder in Google Drive: an images array of shape (4125, 200, 200, 3) and a labels frame of shape (4125, 1), with 964 records in class 0 and 3,161 in class 1.

```
images_path = {
    "/content/drive/MyDrive/"
    "Computer Vision Project/HelmNet_Project/images.npy"
}

labels_path = {
    "/content/drive/MyDrive/"
    "Computer Vision Project/HelmNet_Project/labels-2.csv"
}

images = np.load(images_path)
labels = pd.read_csv(labels_path)

print("Images loaded successfully:", images.shape)
print("Labels loaded successfully:", labels.shape)

print("\nLabel counts:")
print(labels["label"].value_counts().sort_index())
```

```
... Images loaded successfully: (4125, 200, 200, 3)
Labels loaded successfully: (4125, 1)

Label counts:
label
0      964
1     3161
Name: count, dtype: int64
```

```
print(images.shape) #Complete the code to print the shape of the images
print(labels.shape) #Complete the code to print the shape of the labels
```

```
(4125, 200, 200, 3)
(4125, 1)
```

Data Overview (continued)

Random samples of five images from each class were inspected side by side to verify the class definitions and gauge the difficulty of the task. Helmet-class workers wear clearly visible red, yellow, or white helmets, while no-helmet images generally show uncovered heads.

Beyond that, the dataset is highly varied: front, side, and rear views; differing helmet colour, shape, and visibility; wide variation in head size and position within the frame; frames containing more than one person; and lighting and backgrounds shifting across construction sites, factories, and industrial settings, with workers standing, using tools, and in motion.

A meaningful share of images are genuinely difficult — blurred, low-resolution, partially obstructed, or poorly framed, with the worker sometimes occupying only a small part of the frame. One helmet image (index 3892) is blurred to the point that the helmet is hard to distinguish even for a human reviewer. This variability reflects real deployment conditions but raises the expected error rate.

Data Overview: Preparation

Every image was loaded at a uniform 200×200 resolution across three colour channels, giving each example the shape $200 \times 200 \times 3$ expected by the convolutional architectures, with labels held as a single binary column where 1 denotes With Helmet and 0 denotes Without Helmet.

The data was divided into three subsets using a stratified split so class proportions were preserved throughout: 3,300 images for training, 412 for validation, and 413 for the held-out test set. Validation guided architecture comparison and model selection, while the test set was reserved for a single final evaluation so reported test performance remains an honest estimate of unseen-data behaviour.

Pixel values, originally 0–255, were rescaled by dividing by 255 to a 0–1 range, which stabilises training and matches the input expectations of the pretrained VGG16 network. Augmentation — 15-degree rotations, 10% width, height, shear and zoom shifts, and horizontal flips — was applied to training images only, leaving validation and test data unaltered.

EDA: Random Image Samples

This visualization is an exploratory check on the image data. It confirms that the images and the label file were loaded correctly and that each class carries its intended meaning, before any model is trained.

Reading the display: the left column holds images labelled 1, With Helmet; the right column holds images labelled 0, Without Helmet. The number in each title is the image's position in the dataset, not a prediction or a score — “With Helmet (Index: 1092)” simply means image 1092 has a true label of 1.

The code lists the index positions of every label-1 and label-0 image, draws five at random from each list with `random.sample`, and plots the two sets side by side in a two-column grid for direct comparison.



EDA: Sample Observations

The samples on the previous slide indicate the class labels are broadly reasonable: workers in the helmet class wear visible red, yellow, or white safety helmets, while people in the no-helmet class generally have uncovered heads.

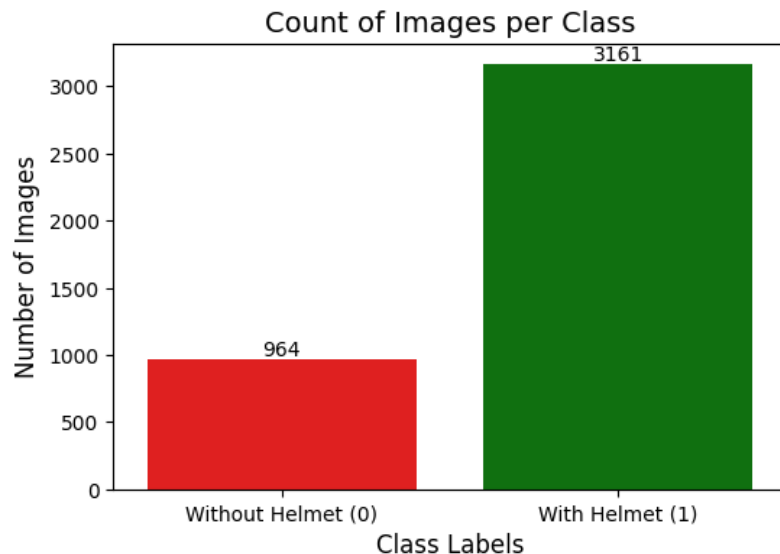
The dataset is visually varied. It includes front, side, and rear views of workers; helmet colours, shapes, and visibility differ; the size and location of each person's head varies considerably; some images contain more than one person; and lighting conditions and backgrounds change across the dataset. Several images are blurred, low-resolution, partially obstructed, or poorly framed, and some workers occupy only a small portion of the frame. The helmet image at index 3892 is particularly blurred, making the helmet difficult to distinguish, and some no-helmet images are similarly affected.

Interpreted correctly, this check does not demonstrate accuracy — no model has been trained at this stage. It establishes only that the image array and label file are aligned, that label 1 means With Helmet and label 0 means Without Helmet, that the data carries considerable visual variability, and that some images would be difficult even for a human to classify. That variability reflects realistic capture conditions but may raise model error where a head or helmet is not clearly visible.

EDA: Class Distribution

The count plot shows how many images belong to each target class. The red bar is label 0, Without Helmet; the green bar is label 1, With Helmet; the number above each bar is the exact image count.

The dataset holds 964 images without helmets and 3,161 with helmets — 4,125 in total. As a share of the whole that is approximately 23.4% without helmets against 76.6% with them, or about 3.28 times as many helmet images as no-helmet images.



EDA: Class Percentages and Method

Class Percentage = (Images in Class ÷ Total Images) × 100

Without Helmet: $964 \div 4,125 \times 100 \approx 23.4\%$

With Helmet: $3,161 \div 4,125 \times 100 \approx 76.6\%$

Imbalance ratio: $3,161 \div 964 \approx 3.28\times$

Together these figures mean that for every no-helmet image the dataset holds roughly three and a quarter helmet images, and that more than three-quarters of all training signal comes from the compliant class.

The chart itself is produced from a 6 × 4-inch figure and a seaborn countplot over the first column of the labels frame, coloured red for class 0 and green for class 1. A loop over the axis patches annotates each bar with its exact count, positioned at the bar's horizontal centre and aligned to its top edge; the remaining calls set the chart title, the axis labels, and the descriptive tick labels “Without Helmet (0)” and “With Helmet (1)”.

EDA: Imbalance and Modelling Implications

The class distribution reveals a pronounced imbalance: With Helmet is the majority class and Without Helmet the minority, with more than three-quarters of the images in the helmet class and less than a quarter in the no-helmet class. A model can exploit that skew — predicting With Helmet for every image would return roughly 76.6% accuracy while identifying not a single unsafe case. Accuracy alone is therefore not an adequate measure of performance.

Two implications follow. The data should be divided using a stratified split so that the training, validation, and test sets preserve approximately the same class proportions. And performance should be judged on macro precision, macro recall, macro F1-score, the confusion matrix, and recall for each class considered separately.

Recall on the Without Helmet class matters most, because a missed violation leaves an unsafe condition unaddressed. Standard binary recall treats label 1 as the positive class, and label 1 here means With Helmet, so it does not measure detection of unsafe cases — class-specific recall and the confusion matrix must be read alongside it. Final model selection should weight macro F1-score and no-helmet recall rather than headline accuracy.

Data Pre-processing: Splitting the Dataset

Splitting the dataset

```
X_train, X_temp, y_train, y_temp = train_test_split(np.array(images), labels, test_size=0.20, random_state=42, stratify=labels) #Complete the code to define the test_size
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp) #Complete the code to define the test_size
```

```
print(X_train.shape, y_train.shape) #Complete the code to print the shape of the train data
print(X_val.shape, y_val.shape) #Complete the code to print the shape of the validation data
print(X_test.shape, y_test.shape) #Complete the code to print the shape of the test data
```

```
(3300, 200, 200, 3) (3300, 1)
(412, 200, 200, 3) (412, 1)
(413, 200, 200, 3) (413, 1)
```

The split was carried out in two stages with scikit-learn's `train_test_split`. The first call held out 20% of the 4,125 images as a temporary pool; the second split that pool in half to produce the validation and test sets. Both calls used `random_state=42` and stratified sampling.

The result is 3,300 training images, 412 validation images, and 413 test images — approximately an 80 / 10 / 10 division. Validation and test differ by a single image because the 825 held-out images cannot be divided evenly into two whole groups.

Data Pre-processing: Split Results and Purpose

Dataset Split Results

The original dataset of **4,125 images** was divided into training, validation, and test datasets.

The resulting shapes are:

```
Training: (3300, 200, 200, 3) (3300, 1)
Validation: (412, 200, 200, 3) (412, 1)
Test:      (413, 200, 200, 3) (413, 1)
```

Each image array follows the structure (number of images, image height, image width, colour channels), so every subset holds 200×200 pixel images with three colour channels. The label arrays carry one binary value per image — 0 for Without Helmet, 1 for With Helmet — and their row counts match the image counts exactly, confirming images and labels stayed aligned through the split.

The subsets serve distinct roles: training updates the model weights, validation guides comparison and model selection, and the test set is held back for a single final evaluation. Using test data to choose between models would leak information and inflate the result.

Data Pre-processing: Stratification

Both splits used stratification — `stratify=labels` on the first call, `stratify=y_temp` on the second.

This keeps the helmet and no-helmet proportions roughly equal across training, validation, and test. It matters because helmet images outnumber no-helmet images by about 3.28 to 1; without it, a subset could badly under-represent the minority class.

`random_state=42` makes the split reproducible, so every model is compared on identical subsets.

Stratification

The split was performed with stratification:

`stratify=labels`

and:

`stratify=y_temp`

Stratification preserves approximately the same helmet and no-helmet class proportions in the training, validation, and test datasets. This is important because the dataset contains more helmet images than no-helmet images.

The use of:

`random_state=42`

makes the split reproducible. Running the code again with the same data and random state should produce the same assignments.

Data Pre-processing: Normalization

Pixel intensities arrive as integers from 0 to 255. Normalisation converts each array to float32 and divides by 255.0, rescaling every value into the range 0 to 1.

The identical constant is applied to the training, validation, and test arrays. Because the divisor is a fixed number rather than a statistic computed from the data, the step introduces no leakage from validation or test into training.

▼ Data Normalization

Since the image pixel values range from 0-255, our method of normalization here will be scaling - we shall divide all the pixel values by 255 to standardize the images to have values between 0-1.

```
11 X_train_normalized = X_train.astype('float32')/255.0 #Complete the code to normalize the training images
    X_val_normalized = X_val.astype('float32')/255.0 #Complete the code to normalize the validation images
    X_test_normalized = X_test.astype('float32')/255.0 #Complete the code to normalize the test images
```

```
11 print(
    "Training range:",
    X_train_normalized.min(),
    "to",
    X_train_normalized.max()
)

print(
    "Validation range:",
    X_val_normalized.min(),
    "to",
    X_val_normalized.max()
)

print(
    "Test range:",
    X_test_normalized.min(),
    "to",
    X_test_normalized.max()
)
```

▼ ... Training range: 0.0 to 1.0
Validation range: 0.0 to 1.0
Test range: 0.0 to 1.0

Data Pre-processing: Normalization Results

Normalization Results

The normalization check produced:

```
Training range: 0.0 to 1.0  
Validation range: 0.0 to 1.0  
Test range: 0.0 to 1.0
```

Before normalization, the image pixels were integers ranging from 0 to 255. Each image array was converted to the float32 data type and divided by 255.0 :

```
X_train_normalized = X_train.astype("float32") / 255.0  
X_val_normalized = X_val.astype("float32") / 255.0  
X_test_normalized = X_test.astype("float32") / 255.0
```

$0 \leq \text{normalized pixel value} \leq 1$

$$0 \div 255 = 0.0$$

$$128 \div 255 \approx 0.502$$

$$255 \div 255 = 1.0$$

The check confirms a 0.0 to 1.0 range in all three subsets — not that every pixel is 0 or 1, but that the smallest value present is 0.0, the largest is 1.0, and the remaining pixels fall between those limits.

Rescaling improves numerical stability, helps the optimiser update weights effectively, and keeps training, validation, and test images on one common scale. With 3,300 / 412 / 413 images at $200 \times 200 \times 3$ and pixels in the 0–1 range, the data is ready for the CNN and transfer-learning models.

Model Building

Four models were built and compared on the same stratified splits, the same normalised inputs, and the same evaluation code, so that differences in performance reflect architecture rather than data handling.

Model 1 is a convolutional neural network trained from scratch — a baseline carrying no prior visual knowledge. Model 2 introduces transfer learning, using the frozen convolutional base of VGG16, pretrained on ImageNet, as a fixed feature extractor feeding a single output neuron. Model 3 keeps that frozen base but replaces the simple classifier with a feed-forward head of 64 and 32 units with 30% dropout. Model 4 trains the Model 3 architecture on augmented images — small rotations, shifts, shear, zoom, and horizontal flips — to widen the effective training distribution.

Each model was judged on validation accuracy, macro recall, macro precision, and macro F1-score, with the confusion matrix and no-helmet recall examined alongside them.

Model Building: Utility Functions

Two helper functions keep evaluation identical across all four models.

`model_performance_classification` converts predicted probabilities to classes at a 0.5 threshold and returns accuracy plus macro recall, macro precision, and macro F1-score in one row; `plot_confusion_matrix` applies the same threshold and renders the TensorFlow confusion matrix as an annotated heatmap.

- Model Building

- Utility Functions

```
[1] # defining a function to compute different metrics to check performance of a classification model built using statsmodels
def model_performance_classification(model, predictors, target):
    """
    Function to compute different metrics to check classification model performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    # checking which probabilities are greater than threshold
    pred = model.predict(predictors).reshape(-1)>0.5
    target = target.to_numpy().reshape(-1)

    acc = accuracy_score(target, pred) # to compute Accuracy
    recall = recall_score(target, pred, average='macro') # to compute Recall
    precision = precision_score(target, pred, average='macro') # to compute Precision
    f1 = f1_score(target, pred, average='macro') # to compute F1-score

    # creating a dataframe of metrics
    df_perf = pd.DataFrame({"Accuracy": acc, "Recall": recall, "Precision": precision, "F1 Score": f1, index=[0]})

    return df_perf

[2] def plot_confusion_matrix(model, predictors, target, ml=False):
    """
    Function to plot the confusion matrix

    model: classifier
    predictors: independent variables
    target: dependent variable
    ml: To specify if the model used is an sklearn ML model or not (True means ML model)
    """

    # checking which probabilities are greater than threshold
    pred = model.predict(predictors).reshape(-1)>0.5
    target = target.to_numpy().reshape(-1)

    # Plotting the Confusion Matrix using confusion matrix() function which is also predefined tensorflow module
    confusion_matrix = tf.math.confusion_matrix(target, pred)
    f, ax = plt.subplots(figsize=(10, 8))
    sns.heatmap(
        confusion_matrix,
        annot=True,
        linewidths=4,
        fmt="d",
        square=True,
        ax=ax
    )
    plt.show()
```

CNN: Architecture and Configuration

Model 1 has two parts: feature-extraction layers — three convolutional blocks with max pooling — and fully connected layers that classify.

Configuration: input $200 \times 200 \times 3$; Conv2D with 16, 32, and 64 filters, 3×3 kernels, ReLU, padding “same”; max pooling 4×4 then 2×2 ; Flatten; Dense(4, ReLU); Dense(1) output. Compiled with Adam (lr 0.001), binary cross-entropy loss, and recall as the tracked metric.

Validity note: the output layer used ReLU. Binary cross-entropy needs a sigmoid output bounded between 0 and 1, so every Model 1 figure that follows is provisional until the layer is changed to sigmoid and the model retrained.

```
# Summary
model_1.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 200, 200, 16)	448
max_pooling2d (MaxPooling2D)	(None, 50, 50, 16)	0
conv2d_1 (Conv2D)	(None, 50, 50, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 25, 25, 32)	0
conv2d_2 (Conv2D)	(None, 25, 25, 64)	18,496
flatten (Flatten)	(None, 40000)	0
dense (Dense)	(None, 4)	168,004
dense_1 (Dense)	(None, 1)	5

Total params: 183,593 (717.16 KB)
Trainable params: 183,593 (717.16 KB)
Non-trainable params: 0 (0.00 B)

CNN: Layer Walkthrough and Parameters

$200 \times 200 \times 16 \rightarrow 50 \times 50 \times 16 \rightarrow 50 \times 50 \times 32 \rightarrow 25 \times 25 \times 32 \rightarrow 25 \times 25 \times 64$

$25 \times 25 \times 64 = 40,000$ features Total parameters: 183,593 (all trainable)

Padding “same” preserves the spatial size through each convolution, so reduction happens only at the pooling layers — 200×200 becomes 50×50 after the 4×4 pool, then 25×25 after the 2×2 pool. Filter depth rises from 16 to 32 to 64, letting deeper layers combine simple edges into shapes, colour patches, head boundaries, and helmet-like structures.

Flatten turns the final $25 \times 25 \times 64$ volume into a 40,000-value vector, which the Dense(4) layer compresses before the single output neuron. The convolutional layers hold only 23,584 parameters while that first dense layer holds 160,004 — roughly 87% of the model’s capacity sits in the classifier rather than the feature extractor, which is a lot of weight resting on four neurons.

With the correct sigmoid output the neuron returns a probability: above 0.5 is read as With Helmet (1), at or below 0.5 as Without Helmet (0). Because the model is trained from scratch, none of these 183,593 weights carry prior visual knowledge — every filter is learned from 3,300 images alone.

CNN: Training Configuration and History

```
history_1 = model_1.fit(
    X_train_normalized, y_train,
    epochs=10, #Complete the code to define the number of epochs
    validation_data=(X_val_normalized, y_val),
    shuffle=True,
    batch_size=32, #Complete the code to define the batch size
    verbose=2
)
```

```
Epoch 1/10
104/104 - 11s - 102ms/step - loss: 0.7070 - recall: 0.8802 - val_loss: 0.4046 - val_recall: 0.9335
Epoch 2/10
104/104 - 3s - 30ms/step - loss: 0.4745 - recall: 0.9027 - val_loss: 0.2523 - val_recall: 0.9525
Epoch 3/10
104/104 - 5s - 46ms/step - loss: 0.3623 - recall: 0.9407 - val_loss: 0.3472 - val_recall: 0.9272
Epoch 4/10
104/104 - 5s - 49ms/step - loss: 0.3750 - recall: 0.9296 - val_loss: 0.2695 - val_recall: 0.9241
Epoch 5/10
104/104 - 3s - 27ms/step - loss: 0.3448 - recall: 0.9355 - val_loss: 0.2725 - val_recall: 0.9589
Epoch 6/10
104/104 - 3s - 25ms/step - loss: 0.3470 - recall: 0.9181 - val_loss: 0.2131 - val_recall: 0.9399
Epoch 7/10
104/104 - 2s - 22ms/step - loss: 0.3360 - recall: 0.9316 - val_loss: 0.2494 - val_recall: 0.9430
Epoch 8/10
104/104 - 3s - 25ms/step - loss: 0.3083 - recall: 0.8936 - val_loss: 0.2754 - val_recall: 0.9557
Epoch 9/10
104/104 - 2s - 22ms/step - loss: 0.3315 - recall: 0.9427 - val_loss: 0.3398 - val_recall: 0.9525
Epoch 10/10
104/104 - 3s - 26ms/step - loss: 0.3071 - recall: 0.9482 - val_loss: 0.2928 - val_recall: 0.9589
```

Training ran for 10 epochs at batch size 32 with shuffling enabled — about 104 steps per epoch over the 3,300 training images, with the 412 validation images scored after each epoch.

Training loss fell steadily from 0.7070 to 0.3071. Validation loss reached its minimum of 0.2131 at epoch 6 and drifted up to 0.2928 by epoch 10 — either normal variation or the first sign of mild overfitting; early stopping would have kept the epoch-6 weights.

CNN: Training and Validation Recall

Training recall climbed from 88.0% in epoch 1 to 94.8% by epoch 10, while validation recall moved from 93.4% to 95.9%. Both stayed high but oscillated from epoch to epoch.

Validation recall sat above training recall for most of the run. That is not necessarily a red flag: training recall accumulates while the weights are still changing during an epoch, whereas validation is measured once at the end with fixed weights.

This Keras recall tracks the helmet class only. It says nothing about no-helmet detection — precisely where the model turns out to be weakest.



CNN: Training and Validation Metrics

```
11 model_1_train_perf = model_performance_classification(model_1, X_train_normalized,y_train)
    print("Train performance metrics")
    print(model_1_train_perf)
```

104/104 ————— 5s 29ms/step
Train performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.930909	0.880991	0.921119	0.898842

```
1 model_1_valid_perf = model_performance_classification(model_1, X_val_normalized,y_val)
    print("Validation performance metrics")
    print(model_1_valid_perf)
```

13/13 ————— 0s 15ms/step
Validation performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.919903	0.875264	0.896007	0.884989

Training reached 93.1% accuracy (macro recall 0.8810, precision 0.9211, F1 0.8988); validation reached 92.0% accuracy (macro recall 0.8753, precision 0.8960, F1 0.8850).

The train-to-validation gaps are small — 0.011 accuracy, 0.006 macro recall, 0.025 macro precision, 0.014 macro F1 — so the model generalised reasonably over 10 epochs. The macro figures already sit below accuracy, the first hint that one class is handled far better than the other.

CNN: Validation Confusion Matrix

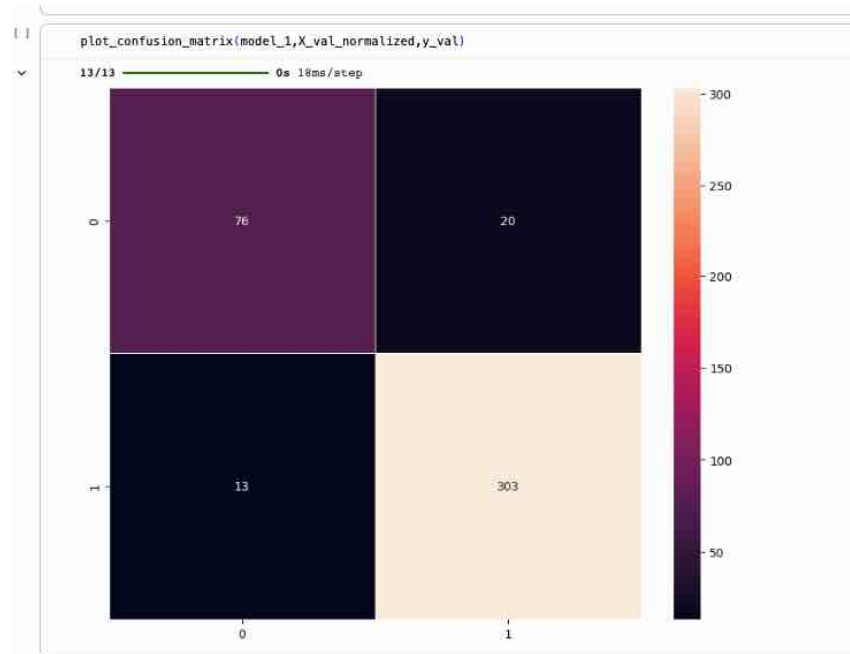
Rows are the true class, columns the prediction. The model made 76 correct no-helmet calls, marked 20 no-helmet images as helmeted, marked 13 helmet images as bare-headed, and made 303 correct helmet calls — 379 right out of 412.

$(76 + 303) \div 412 = 91.99\%$ accuracy

No-helmet recall = $76 \div 96 = 79.2\%$

Helmet recall = $303 \div 316 = 95.9\%$

The 20 false-compliant predictions are the consequential ones: 20.8% of unsafe validation workers would have passed unflagged. A headline 92% accuracy conceals that gap, and the 3.28:1 class imbalance is a likely contributor.

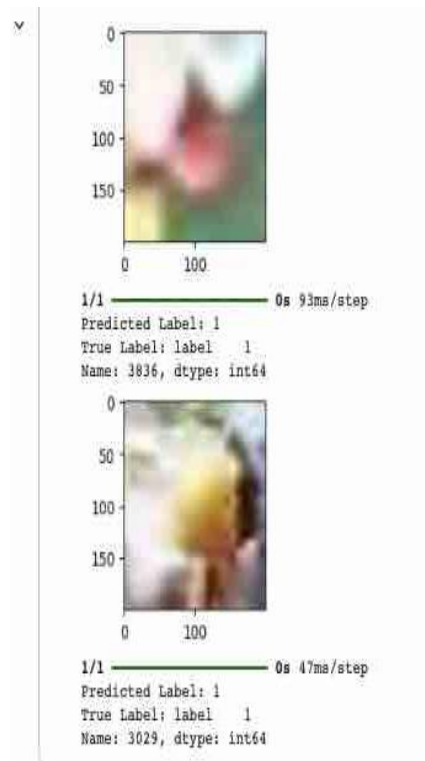


CNN: Example Predictions and Conclusion

Two validation images were inspected individually. Both are heavily blurred — a fair illustration of how hard classification becomes under poor capture conditions.

The example code also carries a bug: the second prediction re-uses index 12 while displaying image 33, comparing one image's prediction with another's label. It must read `X_val_normalized[33]`.

Conclusion — provisional Model 1: validation accuracy 92.0%, macro F1 88.5%, helmet recall 95.9% but no-helmet recall only 79.2%, missing 20 of 96 unsafe cases. Generalisation looks sound, but the class gap is wide and the ReLU output means the model must be rebuilt with a sigmoid output and retrained before a fair comparison.



VGG16 Base: Pretrained Feature Extractor

Model 2 swaps the hand-built feature extractor for VGG16, loaded with ImageNet weights using `include_top=False` and `input_shape=(200, 200, 3)`, so only the convolutional stack is used.

Five blocks of 3×3 convolutions with pooling reduce $200 \times 200 \times 3$ to $6 \times 6 \times 512$ as depth widens $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$. The stack holds 14,714,688 parameters — about $80\times$ the scratch CNN — all learned from ImageNet.

Those weights supply a general vocabulary of edges, textures, and shapes from the start — most valuable when the target dataset is small or imbalanced.

```

vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(200, 200, 3)) #Complete the code to define the shape of the image
vgg_model.summary()

```

Downloading data from https://storage.googleapis.com/tensorflow/tf2/images/vgg16/weights_tf_dim_ordering_tf_kernels_notop.h5
58089254/58089254 On Dva/rtop
Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 200, 200, 3)	0
block1_conv1 (Conv2D)	(None, 200, 200, 64)	1,792
block1_conv2 (Conv2D)	(None, 200, 200, 64)	36,928
block1_pool1 (MaxPooling2D)	(None, 100, 100, 64)	0
block2_conv1 (Conv2D)	(None, 100, 100, 128)	73,856
block2_conv2 (Conv2D)	(None, 100, 100, 128)	147,584
block2_pool1 (MaxPooling2D)	(None, 50, 50, 128)	0
block3_conv1 (Conv2D)	(None, 50, 50, 256)	285,184
block3_conv2 (Conv2D)	(None, 50, 50, 256)	580,800
block3_conv3 (Conv2D)	(None, 50, 50, 256)	580,800
block3_pool1 (MaxPooling2D)	(None, 25, 25, 256)	0
block4_conv1 (Conv2D)	(None, 25, 25, 512)	1,380,160
block4_conv2 (Conv2D)	(None, 25, 25, 512)	2,359,008
block4_conv3 (Conv2D)	(None, 25, 25, 512)	2,359,008
block4_pool1 (MaxPooling2D)	(None, 12, 12, 512)	0
block5_conv1 (Conv2D)	(None, 12, 12, 512)	2,359,008
block5_conv2 (Conv2D)	(None, 12, 12, 512)	2,359,008
block5_conv3 (Conv2D)	(None, 12, 12, 512)	2,359,008
block5_pool1 (MaxPooling2D)	(None, 6, 6, 512)	0

Total params: 14,714,688 (56.13 MB)
Trainable params: 14,714,688 (56.13 MB)
Non-trainable params: 0 (0.00 B)

VGG16 Base: Model 2 Configuration

Every VGG16 layer is frozen with `layer.trainable = False`, so the pretrained filters act as a fixed feature extractor and only the new classifier learns.

Model 2 is a Sequential stack: the VGG16 base, a Flatten turning $6 \times 6 \times 512$ into an 18,432-value vector, and one Dense(1, sigmoid) output. Compiled with Adam at lr 0.001, binary cross-entropy loss, and recall as the tracked metric.

Of 14,733,121 total parameters only 18,433 are trainable — 0.13% of the network. Training is fast and less prone to overfitting than the scratch CNN, but the model can only reweight ImageNet features, not adapt them to helmets.

```

11 # Making all the layers of the VGG model non-trainable. i.e. freezing them
    for layer in vgg_model.layers:
        layer.trainable = False

12 model_2 = Sequential()

    # Adding the convolutional part of the VGG16 model from above
    model_2.add(vgg_model)

    # Flattening the output of the VGG16 model because it is from a convolutional layer
    model_2.add(Flatten())

    # Adding a dense output layer
    model_2.add(Dense(1, activation='sigmoid')) #Complete the code to define the number of neurons in the output layer.

13 opt=Adam(learning_rate=0.001) #Complete the code to define the learning rate
    # Compile model
    model_2.compile(optimizer=opt, loss=keras.losses.BinaryCrossentropy(), metrics=["recall"]) #Complete the code to define the metrics

14 # Generating the summary of the model
    model_2.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 6, 6, 512)	14,714,688
flatten_1 (Flatten)	(None, 18432)	0
dense_2 (Dense)	(None, 1)	18,433

Total params: 14,733,121 (56.28 MB)
 Trainable params: 18,433 (72.00 KB)
 Non-trainable params: 14,714,688 (56.13 MB)

VGG16 Base: Training Setup and History

No augmentation, batch size 16, 10 epochs, $\text{steps_per_epoch} = 3,300 // 16 = 206$, $\text{shuffle}=\text{False}$, validation measured each epoch. Training loss fell $0.3410 \rightarrow 0.1475$ and validation loss $0.2274 \rightarrow 0.1854$ — its lowest of the run, with no upward turn, so no overfitting within these ten epochs.

```
11 train_datagen = ImageDataGenerator()

12 # Epochs
epochs = 10 #Complete the code to define the epochs
# Batch size
batch_size = 16 #Complete the code to define the batch size

history_2 = model_2.fit(train_datagen.flow(X_train_normalized,y_train,
                                         batch_size=batch_size,
                                         seed=42,
                                         shuffle=False),
                        epochs=epochs,
                        steps_per_epoch=X_train_normalized.shape[0] // batch_size,
                        validation_data=(X_val_normalized,y_val),
                        verbose=1)
```

Epoch 1/10
206/206 ————— 20s 92ms/step - loss: 0.3410 - recall: 0.9492 - val_loss: 0.2274 - val_recall: 0.9462
Epoch 2/10
206/206 ————— 2s 11ms/step - loss: 0.4804 - recall: 1.0000 - val_loss: 0.2449 - val_recall: 0.9082
Epoch 3/10
206/206 ————— 19s 92ms/step - loss: 0.2103 - recall: 0.9599 - val_loss: 0.2059 - val_recall: 0.9620
Epoch 4/10
206/206 ————— 2s 9ms/step - loss: 0.1000 - recall: 1.0000 - val_loss: 0.2104 - val_recall: 0.9778
Epoch 5/10
206/206 ————— 17s 81ms/step - loss: 0.1797 - recall: 0.9706 - val_loss: 0.2042 - val_recall: 0.9335
Epoch 6/10
206/206 ————— 2s 9ms/step - loss: 0.1393 - recall: 0.9231 - val_loss: 0.1987 - val_recall: 0.9430
Epoch 7/10
206/206 ————— 16s 79ms/step - loss: 0.1523 - recall: 0.9730 - val_loss: 0.2043 - val_recall: 0.9652
Epoch 8/10
206/206 ————— 2s 9ms/step - loss: 0.1032 - recall: 1.0000 - val_loss: 0.2006 - val_recall: 0.9557
Epoch 9/10
206/206 ————— 17s 82ms/step - loss: 0.1173 - recall: 0.9833 - val_loss: 0.1875 - val_recall: 0.9684
Epoch 10/10
206/206 ————— 2s 10ms/step - loss: 0.1475 - recall: 0.9091 - val_loss: 0.1854 - val_recall: 0.9620

VGG16 Base: Training and Validation Recall

Keras recall swings sharply between epochs: training recall reaches 1.0000 three times yet finishes at 0.9091, while validation recall moves between 0.9082 and 0.9778.

The oscillation follows from a very small trainable head — with 18,433 weights over a frozen feature space, each epoch can shift the decision boundary a long way. Loss, by contrast, fell steadily, so the model was still improving.

Because this metric tracks the helmet class only, high values here say little about no-helmet detection.



VGG16 Base: Performance Metrics

```
model_2_train_perf = model_performance_classification(model_2,X_train_normalized,y_train)

print("Train performance metrics")
print(model_2_train_perf)
```

104/104 ————— 16s 144ms/step
Train performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.974545	0.958599	0.969774	0.964034

```
model_2_valid_perf = model_performance_classification(model_2, X_val_normalized,y_val)

print("Validation performance metrics")
print(model_2_valid_perf)
```

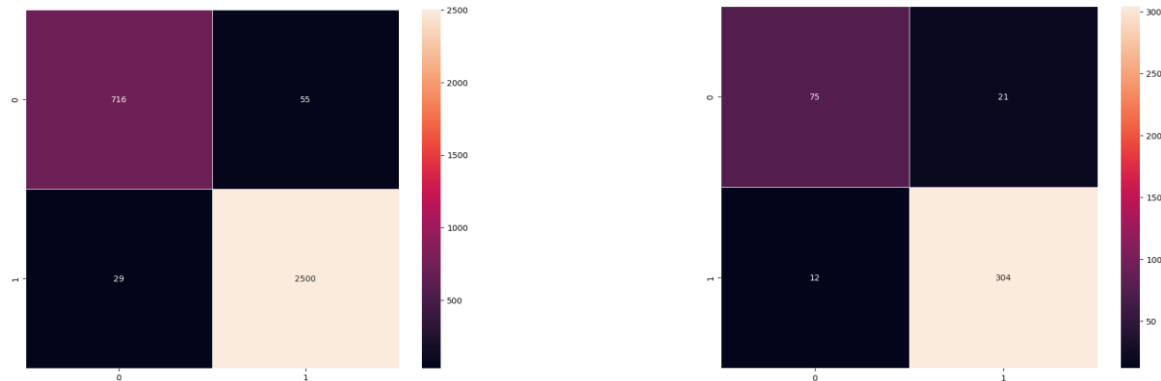
13/13 ————— 2s 148ms/step
Validation performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.919903	0.871638	0.898727	0.884095

Training: accuracy 0.9745, macro recall 0.9586, macro precision 0.9698, macro F1 0.9640.
Validation: accuracy 0.9199, macro recall 0.8716, macro precision 0.8987, macro F1 0.8841.

The train-to-validation gap is wider than the scratch CNN's — roughly 5.5 points of accuracy and 8.7 of macro recall — so freezing the base did not stop the small classifier from fitting the training set more closely than it generalises. Validation accuracy still lands at 0.9199, identical to Model 1.

VGG16 Base: Confusion Matrices



Validation: no-helmet $75 \div 96 = 78.1\%$ · helmet $304 \div 316 = 96.2\%$ · macro $\approx 87.2\%$

Training (left): 716 of 771 no-helmet correct (92.9%) and 2,500 of 2,529 helmet correct (98.9%). Validation (right): 75 of 96 and 304 of 316 — 21 unsafe workers passed as compliant, one more than the scratch CNN missed.

Model 2 matches Model 1's 92.0% validation accuracy with slightly lower macro recall (87.2% vs 87.5%) and higher precision. Transfer learning bought steadier training but did not fix the minority-class weakness — motivating the deeper head in Model 3.

VGG16 + FFNN: Model 3 Configuration

Model 3 keeps the frozen VGG16 base and swaps the single output layer for a feed-forward head: Flatten → Dense(64, ReLU) → Dropout(0.30) → Dense(32, ReLU) → Dense(1, sigmoid).

Configuration: Adam at lr 0.0001 — ten times lower than Models 1 and 2 — binary cross-entropy, batch size 16, 10 epochs, recall tracked, no augmentation. Dropout switches off 30% of inputs each step.

Of 18,260,165 parameters, 1,181,825 are trainable — 64 times Model 2's head — with the 14,714,688 VGG16 weights frozen. Almost all of it sits in the first dense layer (1,179,712).

The generator ran with shuffle=False, so batch composition repeated identically each epoch.

```
model_3.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 6, 6, 512)	14,714,688
flatten_2 (Flatten)	(None, 18432)	0
dense_3 (Dense)	(None, 64)	1,179,712
dropout (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 1)	33

Total params: 18,260,165 (69.66 MB)
 Trainable params: 1,181,825 (4.51 MB)
 Non-trainable params: 14,714,688 (56.13 MB)
 Optimizer params: 2,363,652 (9.02 MB)

VGG16 + FFNN: Training Setup and History

Training used the un-augmented generator with batch size 16, seed 42, shuffle=False, 206 steps per epoch, and 10 epochs. Training loss fell from 0.4477 to 0.2265 and validation loss from 0.3202 to 0.2128, reaching its minimum of 0.2074 at epoch 8 — the small rise afterwards suggests further epochs stopped helping.

```
history_3 = model_3.fit(train_datagen.flow(X_train_normalized,y_train,
                                          batch_size=16, #Complete the code to define the batch size
                                          seed=42,
                                          shuffle=False),
                      epochs=10, #Complete the code to define the number of epochs
                      steps_per_epoch=X_train_normalized.shape[0] // batch_size,
                      validation_data=(X_val_normalized,y_val),
                      verbose=1)
```

Epoch	Time	Step	Loss	Recall	Val_Loss	Val_Recall
Epoch 1/10	206/206	22s	94ms/step	loss: 0.4477	recall: 0.9531	val_loss: 0.3202 - val_recall: 0.9905
Epoch 2/10	206/206	2s	11ms/step	loss: 0.1915	recall: 1.0000	val_loss: 0.3172 - val_recall: 0.9873
Epoch 3/10	206/206	18s	87ms/step	loss: 0.3097	recall: 0.9551	val_loss: 0.2803 - val_recall: 0.9905
Epoch 4/10	206/206	2s	9ms/step	loss: 0.3213	recall: 1.0000	val_loss: 0.2786 - val_recall: 0.9905
Epoch 5/10	206/206	17s	81ms/step	loss: 0.2582	recall: 0.9539	val_loss: 0.2309 - val_recall: 0.9304
Epoch 6/10	206/206	2s	9ms/step	loss: 0.2865	recall: 0.8462	val_loss: 0.2305 - val_recall: 0.9304
Epoch 7/10	206/206	16s	80ms/step	loss: 0.2234	recall: 0.9535	val_loss: 0.2134 - val_recall: 0.9747
Epoch 8/10	206/206	2s	9ms/step	loss: 0.1776	recall: 1.0000	val_loss: 0.2074 - val_recall: 0.9525
Epoch 9/10	206/206	19s	82ms/step	loss: 0.2058	recall: 0.9575	val_loss: 0.2116 - val_recall: 0.9778
Epoch 10/10	206/206	2s	10ms/step	loss: 0.2265	recall: 1.0000	val_loss: 0.2128 - val_recall: 0.9810

VGG16 + FFNN: Training and Validation Recall

Both curves are volatile. Training recall alternates between roughly 0.95 and 1.00 with a sharp dip to 0.846 at epoch 6, while validation recall runs between 0.930 and 0.991 and finishes at 0.981.

With shuffle=False the batch order never changes, and a learning rate of 0.0001 applied to a 1.18-million-weight head moves a threshold-based metric sharply from epoch to epoch. Loss, by contrast, declined steadily.

Because Keras recall scores only the helmet class, these high values conceal the model's weakness on no-helmet images — macro metrics and the confusion matrix give the balanced view.



VGG16 + FFNN: Performance Metrics

```
model_3_train_perf = model_performance_classification(model_3, X_train_normalized, y_train)

print("Train performance metrics")
print(model_3_train_perf)
```

```
104/104 ————— 16s 147ms/step
Train performance metrics
Accuracy  Recall  Precision  F1 Score
0  0.94697  0.905445  0.943736  0.922703
```

```
model_3_valid_perf = model_performance_classification(model_3, X_val_normalized, y_val)

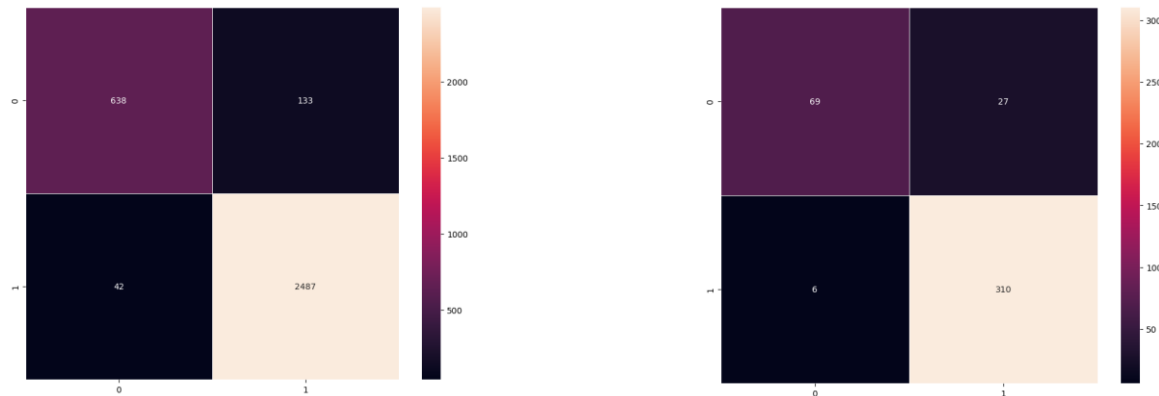
print("Validation performance metrics")
print(model_3_valid_perf)
```

```
13/13 ————— 2s 149ms/step
Validation performance metrics
Accuracy  Recall  Precision  F1 Score
0  0.919903  0.849881  0.919941  0.878241
```

Training: accuracy 0.9470, macro recall 0.9054, macro precision 0.9437, macro F1 0.9227.
Validation: accuracy 0.9199, macro recall 0.8499, macro precision 0.9199, macro F1 0.8782.

Validation trails training by 2.7 points of accuracy but 5.6 of macro recall and 4.5 of macro F1 — moderate overfitting that the 30% dropout did not remove. Validation accuracy again lands on 0.9199, identical to Models 1 and 2, yet macro recall is the lowest of the three.

VGG16 + FFNN: Confusion Matrices



**Validation — no-helmet recall = $69 \div 96 = 71.9\%$ · helmet recall = $310 \div 316 = 98.1\%$
· macro $\approx 85.0\%$**

Training (left) classified 638 of 771 no-helmet images correctly (82.7%) and 2,487 of 2,529 helmet images (98.3%), for 175 errors. Validation (right) gave 69 of 96 and 310 of 316, verifying accuracy as $(69 + 310) \div 412 = 92.0\%$.

The 27 missed no-helmet cases are 28.1% of unsafe validation images — more than one unprotected worker in four goes unflagged. Only six helmet images raised false alarms, so the head bought its precision by leaning toward the With Helmet prediction.

VGG16 + FFNN: Comparison and Conclusion

Model 2 → Model 3 (validation): accuracy 0.9199 both; macro recall 0.8716 → 0.8499; macro precision 0.8987 → 0.9199; unsafe cases missed 21 → 27; false alarms 12 → 6.

The head raised precision and halved false alarms, but only by missing six more unprotected workers — the wrong trade for a safety screen, where a review alert costs far less than an unsafe worker passing as compliant.

Conclusion: Model 3 posts the highest validation macro precision of any model but only 71.9% no-helmet recall and the lowest macro recall, so it should not be chosen on precision alone.

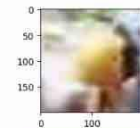
Visualizing the predictions

```
plt.figure(figsize=(2,2))
plt.imshow(X_val[33]) #Complete the code to define the index
plt.show()

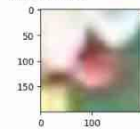
prediction = model_3.predict(X_val_normalized[33].reshape(1,200,200,3)) #Complete the code to define the index
predicted_label = prediction[0][0]>0.5 # Extract the predicted class label
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[33,0] #Complete the code to define the index
print('True Label:', true_label)

plt.figure(figsize=(2,2))
plt.imshow(X_val[12]) #Complete the code to define the index
plt.show()

prediction = model_3.predict(X_val_normalized[12].reshape(1,200,200,3)) #Complete the code to define the index
predicted_label = prediction[0][0]>0.5 # Extract the predicted class label
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[12,0] #Complete the code to define the index
print('True Label:', true_label)
```



1/1 — 0s: 87ms/step
Predicted Label: 1
True Label: 1



1/1 — 0s: 55ms/step
Predicted Label: 1
True Label: 1

VGG16 + FFNN + Augmentation: Configuration

Model 4 reuses the Model 3 architecture — frozen VGG16 base, Flatten, Dense(64, ReLU), Dropout(0.30), Dense(32, ReLU), Dense(1, sigmoid) — and adds data augmentation on the training images.

Configuration: Adam at learning rate 0.0001, binary cross-entropy, batch size 16, recall tracked, 10 epochs per fit call. Labels were converted to float32 arrays before training.

Parameters: 15,896,513 total, of which 1,181,825 are trainable and 14,714,688 frozen — six trainable weight tensors, all in the classification head.

With the base frozen, augmentation is the only remaining lever on generalisation: it changes what the head sees without altering the pretrained filters.

```
model_4.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 5, 5, 512)	14,714,688
flatten_4 (Flatten)	(None, 18432)	0
dense_9 (Dense)	(None, 64)	1,179,712
dropout_2 (Dropout)	(None, 64)	0
dense_10 (Dense)	(None, 32)	2,080
dense_11 (Dense)	(None, 1)	33

Total params: 15,896,513 (68.64 MB)
Trainable params: 1,181,825 (4.51 MB)
Non-trainable params: 14,714,688 (56.13 MB)

Model 4: Data Augmentation

Real-world datasets are rarely large enough to train a CNN from scratch. Augmentation exploits translational invariance — a CNN still recognises a shifted, tilted, or rescaled object.

Applied here: rotation $\pm 15^\circ$, width/height shifts 10%, shear 10%, zoom 10%, horizontal flip on, vertical flip off, fill_mode “nearest”. Upside-down workers do not occur, so vertical flips are excluded.

Batches of 16 are drawn shuffled with seed 42 — (16, 200, 200, 3) images, (16,) labels, float32. Only training images are augmented.

Note: fit() ran twice at 10 epochs, so Model 4 trained 20 epochs — results are provisional.

```
1 train_generator_4 = augmented_train_datagen.flow(  
    X_train_normalized,  
    y_train_array,  
    batch_size=16,  
    seed=42,  
    shuffle=True  
)  
  
1 sample_X, sample_y = next(train_generator_4)  
print("Image batch shape:", sample_X.shape)  
print("Label batch shape:", sample_y.shape)  
print("Image data type:", sample_X.dtype)  
print("Label data type:", sample_y.dtype)  
  
train_generator_4.reset()  
Image batch shape: (16, 200, 200, 3)  
Label batch shape: (16,)  
Image data type: float32  
Label data type: float32  
  
1 print("Optimizer:", model_4.optimizer)  
print("Loss:", model_4.loss)  
print(  
    "Number of trainable weight tensors:",  
    len(model_4.trainable_weights)  
)  
Optimizer: <keras.src.optimizers.adam.Adam object at 0x7983b8692ea8>  
Loss: binary_crossentropy  
Number of trainable weight tensors: 6
```

Model 4: Training History

The augmented generator produced 207 steps per epoch. Over the first ten epochs training loss fell $0.4341 \rightarrow 0.2285$ and validation loss $0.3151 \rightarrow 0.1855$; the second ten-epoch call continued from there, finishing at 0.1725 training and 0.1638 validation loss — the lowest of any model in the study, and validation loss never turned upward.

```
history_4 = model_4.fit(
    train_generator_4,
    epochs=10,
    validation_data=(
        X_val_normalized,
        y_val_array
    ),
    verbose=1
)
```

```
Epoch 1/10
207/207 — 52s 243ms/step - loss: 0.4341 - recall: 0.9636 - val_loss: 0.3151 - val_recall: 0.9905
Epoch 2/10
207/207 — 51s 245ms/step - loss: 0.3411 - recall: 0.9526 - val_loss: 0.2808 - val_recall: 0.9905
Epoch 3/10
207/207 — 49s 236ms/step - loss: 0.3110 - recall: 0.9427 - val_loss: 0.2385 - val_recall: 0.9494
Epoch 4/10
207/207 — 49s 236ms/step - loss: 0.2841 - recall: 0.9454 - val_loss: 0.2241 - val_recall: 0.9462
Epoch 5/10
207/207 — 51s 244ms/step - loss: 0.2725 - recall: 0.9486 - val_loss: 0.2268 - val_recall: 0.8987
Epoch 6/10
207/207 — 49s 237ms/step - loss: 0.2524 - recall: 0.9490 - val_loss: 0.2056 - val_recall: 0.9715
Epoch 7/10
207/207 — 49s 235ms/step - loss: 0.2387 - recall: 0.9490 - val_loss: 0.2072 - val_recall: 0.9747
Epoch 8/10
207/207 — 50s 241ms/step - loss: 0.2384 - recall: 0.9482 - val_loss: 0.1893 - val_recall: 0.9335
Epoch 9/10
207/207 — 53s 255ms/step - loss: 0.2384 - recall: 0.9510 - val_loss: 0.1941 - val_recall: 0.9620
Epoch 10/10
207/207 — 55s 269ms/step - loss: 0.2285 - recall: 0.9529 - val_loss: 0.1855 - val_recall: 0.9557
```

Model 4: Performance Metrics

```
model_4_train_perf = model_performance_classification(model_4, X_train_normalized, y_train)
print("Train performance metrics")
print(model_4_train_perf)
```

104/104 ————— 16s 146ms/step

Train performance metrics				
	Accuracy	Recall	Precision	F1 Score
0	0.952121	0.932247	0.933815	0.933028

```
model_4_valid_perf = model_performance_classification(model_4, X_val_normalized, y_val)
print("Validation performance metrics")
print(model_4_valid_perf)
```

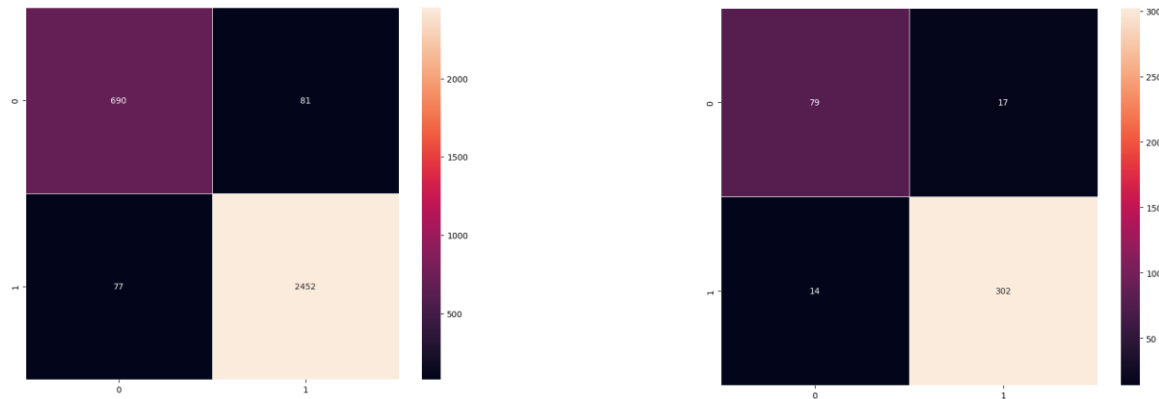
13/13 ————— 2s 146ms/step

Validation performance metrics				
	Accuracy	Recall	Precision	F1 Score
0	0.924757	0.889306	0.898085	0.89358

Training: accuracy 0.9521, macro recall 0.9322, macro precision 0.9338, macro F1 0.9330.
Validation: accuracy 0.9248, macro recall 0.8893, macro precision 0.8981, macro F1 0.8936.

Validation trails training by 2.7 points of accuracy, 4.3 of macro recall, and 3.9 of macro F1 — some overfitting remains, but the gap is narrower than the non-augmented VGG16 models and the macro figures are the most balanced of the four models.

Model 4: Confusion Matrices



Validation — no-helmet $79 \div 96 = 82.3\%$ · helmet $302 \div 316 = 95.6\%$ · macro $\approx 88.9\%$

Training (left): 690 of 771 no-helmet correct (89.5%) and 2,452 of 2,529 helmet correct (97.0%) — 158 errors. Validation (right): 79 of 96 and 302 of 316, giving 381 correct of 412.

Only 17 unsafe workers were mislabelled as compliant — 17.7% of no-helmet validation images, the best of the four models and a marked improvement on Model 3's 28.1%. Augmentation narrowed the class gap rather than simply lifting overall accuracy.

Model 4: Example Predictions

Two blurred validation images were scored individually with the probability printed alongside the label.

Image 12 — helmet probability 0.9357, predicted 1, true 1, correct.

Image 33 — helmet probability 0.9393, predicted 1, true 1, correct.

Both are confident calls on inputs a human would struggle with. Correctness must be judged from the printed labels rather than the picture, and high confidence on degraded frames is itself a caution.

```
# Prediction for validation image 12
plt.figure(figsize=(2, 2))
plt.imshow(X_val[12])
plt.axis('off')
plt.title("Validation Image 12")
plt.show()

prediction = model_4.predict(
    X_val_normalized[12].reshape(
        1, 200, 200, 3
    ),
    verbose=0
)

helmet_probability = float(prediction[0][0])
predicted_label = int(helmet_probability > 0.5)
true_label = int(y_val.iloc[12, 0])

print(
    "Helmet probability:",
    round(helmet_probability, 4)
)
print("Predicted Label:", predicted_label)
print("True Label:", true_label)
print(
    "Prediction Correct:",
    predicted_label == true_label
)
```

Validation Image 12



Helmet probability: 0.9357
Predicted Label: 1
True Label: 1
Prediction Correct: True

```
# Prediction for validation image 33
plt.figure(figsize=(2, 2))
plt.imshow(X_val[33])
plt.axis('off')
plt.title("Validation Image 33")
plt.show()

prediction = model_4.predict(
    X_val_normalized[33].reshape(
        1, 200, 200, 3
    ),
    verbose=0
)

helmet_probability = float(prediction[0][0])
predicted_label = int(helmet_probability > 0.5)
true_label = int(y_val.iloc[33, 0])

print(
    "Helmet probability:",
    round(helmet_probability, 4)
)
print("Predicted Label:", predicted_label)
print("True Label:", true_label)
print(
    "Prediction Correct:",
    predicted_label == true_label
)
```

Validation Image 33



Helmet probability: 0.9393
Predicted Label: 1
True Label: 1
Prediction Correct: True

Model Comparison and Final Selection

models_train_comp_df

	Simple Convolutional Neural Network (CNN)	VGG-16 (Base)	VGG-16 (Base+FFNN)	VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.930909	0.974545	0.946970	0.952121
Recall	0.880991	0.958599	0.905445	0.932247
Precision	0.921119	0.969774	0.943736	0.933815
F1 Score	0.898842	0.964034	0.922703	0.933028

Next steps: [Generate code with models_train_comp_df](#) [New interactive sheet](#)

Validation macro F1 — CNN 0.8850 · VGG16 Base 0.8841 · +FFNN 0.8782 · +Augmentation 0.8936

The table above compares training performance. On validation, all four models sit at 0.9199 accuracy except Model 4 at 0.9248, and macro recall separates them further — 0.8753, 0.8716, 0.8499, 0.8893. Model 4 leads on accuracy, macro recall, and macro F1; Model 3 holds the highest precision but is the least balanced.

Model 4 — VGG-16 (Base + FFNN + Data Aug) — was therefore selected and carried to a single evaluation on the held-out test set.

Model 4: Test Performance and Recommendations

**Test — no-helmet $76 \div 97 = 78.4\%$ · helmet $292 \div 316 = 92.4\%$ · macro $\approx 85.4\%$
 $21 \div 97 \times 100 \approx 21.6\%$ of unsafe test cases missed**

On the held-out test set Model 4 scored 0.8910 accuracy, 0.8538 macro recall, 0.8465 macro precision, and 0.8500 macro F1 — 368 correct of 413, from a confusion matrix of 76 / 21 / 24 / 292.

Test results sit three to five points below validation (accuracy -0.0337, macro recall -0.0355, macro precision -0.0516, macro F1 -0.0436), so validation somewhat overestimated performance — though test accuracy still clears the 76.6% majority-class baseline.

Recommendations: rebuild and train once for 10 epochs for a controlled comparison; tune the decision threshold on validation data; collect more no-helmet, blurred, distant, and low-light images; route low-confidence predictions to human review; validate across sites, cameras, and helmet styles; monitor no-helmet recall in production; and deploy as decision support, not autonomous enforcement.

Model Comparison: Training and Validation

The two comparison tables place all four models side by side. On the training data VGG-16 (Base) leads every metric — 0.9745 accuracy and 0.9586 macro recall — but training performance measures memorisation as much as skill.

On validation the ranking inverts: three of the four models tie at exactly 0.9199 accuracy, and only VGG-16 (Base+FFNN+Data Aug) breaks away at 0.9248, with the highest macro recall (0.8893) and macro F1 (0.8936).

models_train_comp_df	
...	Simple Convolutional Neural Network (CNN) VGG-16 (Base) VGG-16 (Base+FFNN) VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.930909 0.974545 0.946970 0.952121
Recall	0.880991 0.958599 0.905445 0.932247
Precision	0.921119 0.969774 0.943736 0.933815
F1 Score	0.898842 0.964034 0.922703 0.933028
Next steps:	Generate code with models_train_comp_df New interactive sheet

models_valid_comp_df	
...	Simple Convolutional Neural Network (CNN) VGG-16 (Base) VGG-16 (Base+FFNN) VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.919903 0.919903 0.919903 0.924757
Recall	0.875264 0.871638 0.849881 0.889306
Precision	0.896007 0.898727 0.919941 0.898085
F1 Score	0.884989 0.884095 0.878241 0.893580
Next steps:	Generate code with models_valid_comp_df New interactive sheet

Model Comparison: Generalisation Gap

Subtracting validation from training exposes how much of each model's skill is memorisation. The CNN from scratch has the smallest gap (0.011 accuracy, 0.006 macro recall) but the weakest ceiling. VGG-16 (Base) has the widest (0.055 accuracy, 0.087 macro recall) — its tiny 18k-parameter head overfits the frozen feature space quickly.

Adding the FFNN head halves the recall gap to 0.056, and augmentation narrows it further to 0.043 while simultaneously lifting validation scores — the only change that improved both sides of the ledger at once.

models_valid_comp_df

	Simple Convolutional Neural Network (CNN)	VGG-16 (Base)	VGG-16 (Base+FFNN)	VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.919903	0.919903	0.919903	0.924757
Recall	0.875264	0.871638	0.849881	0.889306
Precision	0.896007	0.898727	0.919941	0.898085
F1 Score	0.884989	0.884095	0.878241	0.893580

Next steps:
[Generate code with models_valid_comp_df](#)
[New interactive sheet](#)

models_train_comp_df - models_valid_comp_df

	Simple Convolutional Neural Network (CNN)	VGG-16 (Base)	VGG-16 (Base+FFNN)	VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.011006	0.054643	0.027067	0.027364
Recall	0.005728	0.086961	0.055563	0.042941
Precision	0.025113	0.071047	0.023795	0.035730
F1 Score	0.013852	0.079939	0.044462	0.039448

Final Model Selection

Selection used validation evidence only. Model 4 posts the highest validation accuracy (0.9248), macro recall (0.8893), and macro F1 (0.8936). Model 3 holds the highest macro precision (0.9199) but the lowest macro recall (0.8499) — it buys precision by under-calling the minority class.

For a safety screen, macro F1 and no-helmet recall outrank precision: an extra review alert is cheap, a missed unprotected worker is not. Model 4 wins outright on that criterion.

The choice stays provisional — Model 1 used a ReLU output and Model 4 accumulated 20 epochs across two fit() calls, so both need rebuilding before the comparison is final.

Test Performance

models_valid_comp_df

	Simple Convolutional Neural Network (CNN)	VGG-16 (Base)	VGG-16 (Base+FFNN)	VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.919903	0.919903	0.919903	0.924757
Recall	0.875264	0.871638	0.849881	0.889306
Precision	0.896007	0.898727	0.919941	0.898085
F1 Score	0.884989	0.884095	0.878241	0.893580

Next steps: [Generate code with models_valid_comp_df](#) [New interactive sheet](#)

Test Performance of the Selected Model

With the choice locked on validation, Model 4 was evaluated once on the untouched test set: accuracy 0.8910, macro recall 0.8538, macro precision 0.8465, macro F1 0.8500 — 368 correct of 413 images, 45 errors.

Every metric falls three to five points below validation (accuracy -0.0337, macro recall -0.0355, macro precision -0.0516, macro F1 -0.0436), so validation modestly overestimated real-world performance — though test accuracy still clears the 76.6% majority-class baseline by more than twelve points.

```
1) # Assign the selected validation model
   best_model = model_4
   best_model_name = "VGG-16 (Base+FPNN+Data Aug)"

   # Evaluate it on the test dataset
   model_test_perf = model_performance_classification(
       best_model,
       X_test_normalized,
       y_test
   )

   print("Selected model:", best_model_name)
   print("Test performance metrics:")
   print(model_test_perf)

13/13 ————— 3s 162ms/step
Selected model: VGG-16 (Base+FPNN+Data Aug)
Test performance metrics:
  Accuracy  Recall  Precision  F1 Score
0  0.891041  0.853778  0.846454  0.850016
```

Test Confusion Matrix and Conclusion

No-helmet $76 \div 97 = 78.4\%$

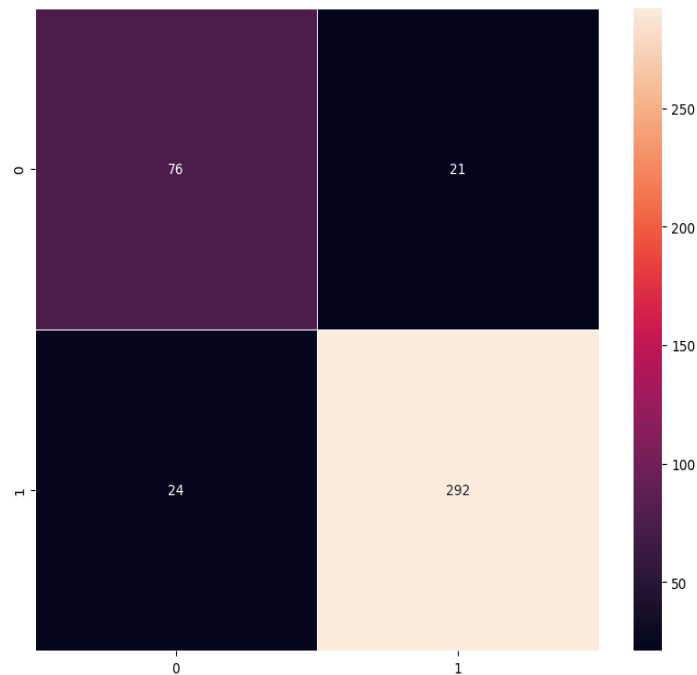
Helmet $292 \div 316 = 92.4\%$

Macro $\approx 85.4\%$ • $21 \div 97 \approx 21.6\%$ missed

The matrix reads 76 / 21 / 24 / 292 — rows are actual class, columns predicted: 76 unprotected workers flagged, 21 missed, 24 false alarms, 292 correct.

At the 0.5 threshold about one no-helmet image in five passes as compliant. False alarms cost review time; misses cost safety — hence threshold tuning on validation data as the first fix.

Model 4 is the best of the four and works as a supervised screening tool, but 78.4% no-helmet recall is not enough for autonomous enforcement.



Rows = actual class, columns = predicted (0 = no helmet, 1 = helmet)

Data Background and Contents

SafeGuard Corp operates in hazardous environments — construction sites, factories, industrial plants — where the safety helmet is the main defence against head injury from falling objects and machinery. Compliance is enforced by supervisors watching workers in person, which does not scale across large sites and is least reliable when activity is highest.

The dataset holds 4,125 labelled images, supplied as a NumPy array (images.npy) with a matching label file (labels-2.csv). Each image is 200×200 pixels across three RGB channels and carries one binary label: 1 for With Helmet (3,161 images, 76.6%) and 0 for Without Helmet (964 images, 23.4%) — a 3.28:1 imbalance.

Content mirrors live capture conditions: front, side, and rear views; helmets differing in colour, shape, and visibility; workers standing, using tools, and moving; single and multi-person frames; and wide variation in lighting, background, subject size, and camera distance. A meaningful minority are blurred, obstructed, or poorly framed.

That realism is both the value and the limit of the data: it trains the model for conditions the system will actually meet, but caps achievable accuracy, since some frames are ambiguous even to a human reviewer.

Actionable Insights: Provisional Model Selection

Validation — accuracy 0.9248 · macro recall 0.8893 · macro precision 0.8981 · macro F1 0.8936

Model 4, the VGG16 base with an FFNN head and data augmentation, produced the strongest available validation results. It leads on accuracy, macro recall, and macro F1-score, giving the best balance between overall classification performance and class-specific recall.

The selection is provisional for two reasons. Model 1 was trained with a ReLU output instead of the sigmoid activation required for binary classification, and Model 4 accumulated 20 epochs across two consecutive `fit()` calls while every other model was trained for 10.

A controlled comparison therefore requires Model 1 rebuilt with `Dense(1, activation="sigmoid")` and Model 4 rebuilt and trained once for 10 epochs. Architecture and hyperparameters must not be adjusted in response to test results already observed — that would leak test information into model design.

Provisional Test Performance

Test — accuracy 0.8910 · macro recall 0.8538 · macro precision 0.8465 · macro F1 0.8500

Model 4 classified 368 of 413 test images correctly and made 45 errors. Test accuracy clears the 76.6% majority-class baseline by more than twelve points, so the model is doing substantially more than exploiting the class imbalance.

Every metric nonetheless declined against validation: accuracy -0.0337, macro recall -0.0355, macro precision -0.0516, macro F1 -0.0436 — roughly three to five percentage points.

Validation therefore somewhat overestimated performance on the independent test set. That is the expected direction, since validation guided model selection, but it is a reminder that validation numbers are not a deployment forecast and that the test figures are the more honest estimate of unseen-data behaviour.

Safety Implications

No-helmet recall = $76 \div 97 = 78.4\%$ · Helmet recall = $292 \div 316 = 92.4\%$

$21 \div 97 \approx 21.6\%$ of unsafe test cases declared compliant

Overall accuracy must not be read alone, because helmet images outnumber no-helmet images by more than three to one. The test confusion matrix reads 76 correct no-helmet calls, 21 no-helmet images classified as helmeted, 24 helmet images classified as bare-headed, and 292 correct helmet calls.

The 21 false-compliance errors are the consequential ones: an unsafe condition goes undetected and no intervention is triggered. The 24 false alarms cost review time and erode trust in the alerts, but they are far less safety-critical than declaring an unprotected worker compliant.

Primary operational finding: at the default probability threshold of 0.5, the model fails to flag roughly one in every five no-helmet images. That error rate is too high for the model to serve as the sole safety-enforcement mechanism.

Key Insights (1–3)

1. Overall accuracy overstates operational effectiveness. 89.1% test accuracy sits beside 78.4% no-helmet recall: the class imbalance lets headline accuracy stay high while a meaningful share of unsafe cases is missed. Evaluation should prioritise no-helmet recall, macro recall, macro F1-score, the count of unsafe false-compliance errors, and train-validation-test consistency.

2. Standard Keras recall measures the less critical class. Because label 1 is With Helmet, Keras recall reports helmet recall and says nothing directly about detecting bare-headed workers. Helmet recall, no-helmet recall, macro recall, and confusion matrices should all continue to be reported side by side.

3. Augmentation helped but did not close the safety gap. Model 4 achieved the strongest validation macro F1-score, so augmentation did improve generalisation across variations in position, framing, and orientation — yet no-helmet performance remained materially weaker than helmet performance in both validation and test.

Key Insights (4–5)

4. Image quality is a material source of risk. The dataset contains blurred, obstructed, distant, poorly framed, and low-resolution images in which helmet status is difficult to determine even for a human reviewer. Image-quality checks should be built into any future system so that unreadable frames are never treated as confident compliance decisions.

5. Whole-image classification may not be sufficient for deployment. The current models label an entire image rather than locating individual workers, so they cannot assign helmet status person by person. That breaks down when a frame contains several workers, when some are compliant and others are not, when a helmet appears in the background, or when the relevant worker occupies only a small part of the image.

Implication. A production system will likely need person detection followed by per-person helmet classification, or an object-detection model that localises both workers and helmets in the same pass.

Recommendations (1–4)

- 1. Correct and rerun the comparison.** Rebuild Model 1 with Dense(1, activation="sigmoid"), rebuild Model 4 from its initial weights and train it once for 10 epochs, regenerate all training and validation metrics and comparison tables, and confirm the final model on validation results only.
- 2. Prioritise no-helmet recall in selection.** Where macro F1-scores are close, prefer the model that detects more no-helmet images, misclassifies fewer unprotected workers as compliant, holds an acceptable false-alarm rate, and shows a smaller generalisation gap.
- 3. Evaluate a more conservative threshold.** At 0.5 the model declares compliance whenever helmet probability exceeds one half. A higher bar would demand stronger evidence of compliance, raising no-helmet recall at the cost of more false alarms. Thresholds must be chosen on validation data, never on test.
- 4. Address the class imbalance.** Collect additional no-helmet images, and consider class-weighted binary cross-entropy, balanced batch sampling, controlled minority-class augmentation, and class-specific reporting after every experiment.

Recommendations (5–9)

5. Improve data and label quality. Collect more low-light, blurred, occluded, distant, side- and rear-view images, varied helmet colours and styles, different sites and camera systems, and multi-worker frames; re-check ambiguous images for labelling accuracy.

6. Route uncertain cases to human review. Escalate when helmet probability sits near the threshold, image quality is poor, several workers appear, a head is obstructed, or the model and an image-quality check disagree.

7. Validate across operational environments. Test across different sites, camera positions and resolutions, indoor and outdoor lighting, weather, worker populations, and helmet designs. Performance on one dataset does not establish reliability everywhere.

8. Monitor after deployment. Track no-helmet recall, unsafe false-compliance rate, false-alarm rate, confidence distribution, image-quality failures, drift over time, and differences across sites, with periodic human audits to signal retraining.

9. Maintain privacy and governance controls. Access controls, defined retention periods, secure storage, documented authorised uses, worker notification where required, and periodic privacy reviews.

Final Conclusion

The project demonstrates that CNNs and transfer learning can classify helmet use from workplace images. Model 4 produced the strongest provisional validation performance and reached 89.1% test accuracy with an 85.0% macro F1-score — comfortably above the 76.6% majority-class baseline.

It nonetheless identified only 78.4% of no-helmet test images, classifying 21 of 97 unsafe cases as helmet-compliant. Strong overall accuracy does not guarantee reliable safety performance: the model is a credible screening and decision-support tool, but it is not ready to act as an autonomous safety-enforcement system.

Any deployment decision should be judged on no-helmet recall, unsafe false-compliance errors, balanced class performance, external validation, human review, continuous monitoring, and privacy and governance controls.

Recommendation: use the model only as a human-supervised screening tool while improving minority-class representation, evaluating a more conservative compliance threshold, expanding person-level detection, and validating across operational environments.



Power Ahead!

